Laborator 1: Introducere în Fusion 360 pentru Proiectarea PCB-urilor

Obiective:
- Familiarizarea studenților cu interfața și funcționalitățile de bază ale Fusion 360.
- Învățarea pasilor necesari pentru a începe un nou proiect de PCB în Fusion 360.
- Crearea unui circuit simplu pentru a înțelege fluxul de proiectare de la schematic la board layout.

Activități Principale:

1. Configurarea Mediului de Lucru:
   - Studenții sunt instruiți cum să configureze spațiul de lucru în Fusion 360 pentru proiectare PCB.
   - Revizuirea diferitelor module și unelte disponibile în Fusion 360 specific pentru PCB design, incluzând schematics, PCB layout, și 3D PCB view.

2. Crearea unui Proiect Nou:
   - Se discută despre cum să se inițieze un proiect nou, setările inițiale necesare, și importanța organizării fișierelor și a folderelor.
   - Studenții creează un nou fișier de schematic și încep să adauge componentele electronice utilizând biblioteca integrată.

3. Desenarea Schematicului:
   - Învățarea metodelor de plasare a componentelor în schematic.
   - Conectarea componentelor folosind fire și noduri, demonstrând importanța unei rutări corecte a semnalelor pentru a preveni erorile de design.
   - Verificarea schematicului pentru erori folosind Electrical Rule Check (ERC) pentru a asigura că toate conexiunile și componentele sunt configurate corect.

4. Generarea PCB-ului din Schematic:
   - Transferul designului din schematic la PCB layout.
   - Aranjarea componentelor pe board și utilizarea uneltelor de auto-route pentru a simplifica conexiunile între pini.
   - Introducerea noțiunilor de design rule check (DRC) pentru a verifica conformitatea fizică a designului PCB cu standardele industriei.

5. Simularea și Verificarea Designului:
   - Utilizarea uneltelor de simulare pentru a testa funcționalitatea circuitului înainte de fabricație.
   - Discuții despre importanța testării și validării designului înainte de a proceda la producția fizică a PCB-ului.

6. Exportul Designului pentru Fabricație:

- Prepararea fișierelor pentru fabricație, inclusiv generarea de Gerber files, care sunt standardul industriei pentru producția de PCB-uri.
- Discuții despre cum să comunici eficient cu fabricanții de PCB-uri și ce informații sunt necesare pentru a asigura o producție fără erori.

Concluzii:
- Laboratorul se încheie cu o discuție despre importanța unui workflow eficient și a unor bune practici în designul de PCB-uri.
- Studenții sunt încurajați să exploreze mai departe avansatele capacități de design și simulare ale Fusion 360.

Concepte Esențiale din Laboratorul 1: Fusion 360 pentru Proiectarea PCB-urilor

1. Interfața Fusion 360:
   - Familiarizarea cu interfața și modulele specifice pentru proiectarea PCB-urilor, incluzând schematics, PCB layout, și 3D PCB view.

2. Proiectarea Schematicului:
   - Crearea și gestionarea schematicelor care reprezintă diagrama electrică a PCB-ului.
   - Utilizarea corectă a tool-urilor pentru plasarea și conectarea componentelor electronice.

3. Generarea Layout-ului PCB:
   - Transferul designului din schematic la PCB layout.
   - Aranjarea componentelor și utilizarea rutelor pentru a conecta electric componentele conform schematicului.

4. Verificarea Designului:
   - Aplicarea și înțelegerea Electrical Rule Check (ERC) pentru a identifica și corecta orice erori electrice în schematic.
   - Utilizarea Design Rule Check (DRC) pentru a asigura că layout-ul PCB-ului respectă toate specificațiile tehnice necesare pentru fabricație.

5. Exportul pentru Fabricație:
   - Pregătirea și exportul fișierelor necesare pentru fabricația PCB-ului, inclusiv generarea de Gerber files, care sunt standardul industriei.

Laborator 2: Crearea și Gestionarea Componentelor Personalizate în Fusion 360

Obiective:
- Învățarea gestionării și creării de componente personalizate în biblioteca Fusion 360.
- Familiarizarea cu procesul de definire a simbolurilor, footprint-urilor și a parametrilor pentru componente noi.

Activități Principale:

1. Introducere în Bibliotecile de Componente:
   - Discuții despre importanța bibliotecilor de componente în designul de PCB și cum pot fi acestea gestionate în Fusion 360.
   - Explorarea interfeței bibliotecii și învățarea despre categoriile de componente disponibile și cum pot fi acestea organizate eficient.

2. Crearea de Componente Noi:
   - Instruirea pas cu pas în procesul de creare a unui nou simbol de componentă. Aceasta include desenarea simbolului schematic în editorul grafic.
   - Definirea footprint-ului (package) pentru componenta nouă, inclusiv plasarea pad-urilor și configurarea dimensiunilor fizice.

3. Definirea și Asocierea unui Device:
   - Explicarea conceptului de 'Device' în Fusion 360, care leagă simbolul schematic și footprint-ul într-o singură entitate utilizabilă.
   - Demonstrarea procesului de asociere a simbolului cu footprint-ul și adăugarea de informații suplimentare, cum ar fi parametrii electrici și descrierea componentei.

4. Verificarea și Validarea Componentelor Create:
   - Utilizarea uneltelor de verificare pentru a asigura că noile componente sunt corect definite și compatibile cu nevoile designului.
   - Discuții despre importanța testării componentelor în cadrul unui circuit de test pentru a evalua performanța și funcționalitatea.

5. Gestionarea și Actualizarea Bibliotecii de Componente:
   - Învățarea despre cum să actualizezi și să menții biblioteca de componente pentru a reflecta schimbările din specificațiile componente sau noi adăugiri.
   - Strategii de partajare și colaborare pe biblioteci de componente cu alte echipe sau membri ai proiectului.

6. Exportul și Integrarea Componentelor în Proiecte Mai Mari:
   - Demonstrarea exportului componentelor personalizate pentru a fi folosite în alte proiecte de PCB sau pentru a fi partajate cu alți utilizatori.
   - Discuții despre cele mai bune practici pentru integrarea eficientă a componentelor noi în proiecte de design complexe.

Concluzii:
- Laboratorul se încheie cu o evaluare a importanței unei biblioteci bine gestionate de componente pentru eficiența și succesul în proiectarea de PCB-uri.
- Studenții sunt încurajați să continue explorarea și extinderea bibliotecilor lor de componente în proiecte viitoare.

Concepte Esențiale din Laboratorul 2: Crearea și Gestionarea Componentelor Personalizate în Fusion 360

1. Biblioteca de Componente:
   - Importanța Bibliotecilor: Element central în gestionarea eficientă a componentelor utilizate în proiectele de PCB. O bibliotecă bine organizată facilitează accesul rapid la componente și îmbunătățește eficiența în proiectare.
   - Gestionarea Bibliotecilor: Capacitatea de a adăuga, edita și organiza componentele în biblioteci este crucială pentru menținerea unui flux de lucru ordonat și eficient.

2. Crearea de Componente:
   - Simboluri: Designul simbolurilor schematicului pentru reprezentarea grafică a componentelor în diagramele electrice.
   - Footprints (Package): Crearea footprints pentru componente, definind dimensiunile fizice și configurarea pinilor, esențială pentru asamblarea PCB-ului.

3. Device-urile în Fusion 360:
   - Definirea unui Device: Un device în Fusion 360 este o entitate care leagă simbolurile și footprints, reprezentând o componentă completă și utilizabilă în designurile PCB.
   - Asocierea Componentelor: Procesul de legare a simbolului schematic la footprint-ul său, asigurând că toate aspectele designului componentei sunt sincronizate și corect configurate.

4. Verificarea și Validarea Componentelor:
   - Testarea Componentelor: Verificarea că noile componente funcționează așa cum este prevăzut în cadrul circuitului complet, folosind simulări și teste de validare.
   - Erori de Design: Utilizarea tool-urilor de verificare pentru a detecta și corecta erorile înainte de a finaliza designul PCB.

5. Exportul și Integrarea Componentelor:
   - Utilizarea în Proiecte: Abilitatea de a exporta și a reintegra componente personalizate în diferite proiecte de PCB este esențială pentru eficiență și reutilizabilitate.

Laborator 3: Optimizarea Layout-ului PCB și Gestionarea Planurilor de Masă în Fusion 360

Obiective:
- Învățarea tehnicilor avansate de optimizare a layout-ului PCB pentru a îmbunătăți performanța și eficiența proiectelor.
- Familiarizarea cu gestionarea și implementarea planurilor de masă în designul PCB.

Activități Principale:

1. Introducere în Optimizarea Layout-ului PCB:
   - Prezentarea principiilor de bază ale unui layout eficient de PCB, inclusiv considerații privind rutarea traseelor, plasarea componentelor și gestionarea spațiului.
   - Discuții despre impactul layout-ului pe performanța electronică și mecanică a PCB-ului.

2. Tehnici de Rutare a Traseelor:
   - Instruirea studenților în tehnici de rutare manuală și automată, cu accent pe evitarea interferențelor și minimizarea zgomotului.
   - Explorarea uneltelor avansate din Fusion 360 care ajută la optimizarea rutării, inclusiv ajustarea lățimii traseelor și utilizarea diferitelor straturi.

3. Implementarea Planurilor de Masă:
   - Explicarea importanței planurilor de masă în reducerea zgomotului și a interferențelor în circuit.
   - Demonstrarea creării și configurării unui plan de masă în Fusion 360, inclusiv plasarea acestuia în layout și conectarea la masele componentelor.

4. Gestionarea Layerelor și Zonelor de Masă:
   - Discuții despre cum să se gestioneze eficient layerele în Fusion 360 pentru a optimiza performanța electrică și structurală a PCB-ului.
   - Învățarea despre separarea și izolarea zonelor de masă pentru a minimiza impactul semnalelor de înaltă frecvență asupra componentelor sensibile.

5. Optimizare și Simulare:
   - Utilizarea simulărilor pentru a evalua eficiența layout-ului propus, cu un focus pe detectarea și rezolvarea potențialelor probleme de design.
   - Modificarea și ajustarea design-ului în funcție de rezultatele simulărilor pentru a atinge performanța dorită.

6. Revizuire și Finalizare a Designului:
   - Etapele finale de verificare a design-ului PCB, inclusiv verificarea regulilor de design (DRC) și pregătirea setului de documente pentru fabricație.
   - Discuții despre cele mai bune practici pentru finalizarea și trimiterea design-ului la producție.

Concluzii:
- Laboratorul subliniază importanța unei abordări atente și detaliate în optimizarea layout-ului PCB pentru a asigura funcționarea optimă a circuitelor.
- Studenții sunt încurajați să aplice cunoștințele dobândite în proiecte reale și să exploreze mai departe capacitățile avansate de design ale Fusion 360.

Concepte Esențiale din Laboratorul 3

1. Optimizarea Layout-ului PCB

- Plasarea Componentelor: Este crucială pentru optimizarea performanței generale a PCB-ului. Componentele ar trebui plasate astfel încât să minimizeze lungimea traseelor și să evite interferențele electromagnetice.
- Rutarea Traseelor: Traseele trebuie rulate eficient pentru a reduce rezistența și susceptibilitatea la interferențe, utilizând tehnici precum rutarea pe straturi multiple și evitarea traseelor lungi și încrucișate.

2. Gestionarea Planurilor de Masă
- Crearea Planurilor de Masă: Planurile de masă sunt esențiale pentru reducerea zgomotului și a interferențelor RF. Ele trebuie să fie ample și bine conectate la masele componentelor pentru a asigura o referință de masă stabilă și pentru a sprijini decuplarea.
- Conexiuni la Masă: O bună practică este utilizarea „vias" multiple pentru a conecta planurile de masă de pe straturile diferite, asigurând o impedanță de masă scăzută și o integritate bună a semnalului.

3. Simulare și Verificare
- Simulările de Integritate a Semnalului: Acestea sunt esențiale pentru a valida performanța circuitelor de înaltă viteză, verificând că semnalele îndeplinesc specificațiile necesare fără distorsiuni semnificative.
- Design Rule Check (DRC): Verificarea regulilor de design este critică pentru a asigura că layout-ul PCB respectă toate standardele tehnice și specificațiile de fabricație necesare pentru producție.

Laborator 4: Fundamentele Git pentru Controlul Versiunilor

Obiective:
- Familiarizarea studenților cu conceptele de bază și comenzi Git pentru gestionarea eficientă a versiunilor de cod.
- Învățarea fluxurilor de lucru Git pentru colaborare în echipe de dezvoltare.

Activități Principale:

1. Introducere în Git:
- Explicarea importanței controlului versiunilor și a rolului Git în gestionarea codului sursă.
- Prezentarea arhitecturii și conceptelor de bază din Git, incluzând repository-uri, commit-uri, branch-uri și merge-uri.

2. Configurarea Mediului Git:
- Instrucțiuni pas cu pas pentru configurarea Git pe sistemele locale ale studenților.
- Crearea unui cont pe GitHub și configurarea conexiunii între mediul local și GitHub.

3. Crearea și Gestionarea Repository-urilor:
  - Demonstrarea creării unui nou repository Git local și pe GitHub.
  - Practicarea clonării unui repository și gestionarea fișierelor (adăugare, modificare, ștergere).

4. Folosirea Branch-urilor și Merge-urilor:
  - Învățarea cum să creeze și să gestioneze branch-uri în Git pentru a permite lucrul simultan pe multiple caracteristici ale proiectului.
  - Exerciții de merge pentru a integra modificările din diferite branch-uri, discutând potențialele conflicte și cum să le rezolve.

5. Comenzi Git Esențiale:
  - Examinarea și practicarea unor comenzi Git fundamentale: `git add`, `git commit`, `git push`, `git pull`, `git fetch`, și `git merge`.
  - Utilizarea comenzi avansate precum `git rebase` pentru a curăța istoricul de commit-uri.

6. Colaborare și Fluxuri de Lucru:
  - Explorarea diferitelor modele de colaborare în Git, inclusiv fluxul de lucru centralizat și Git Flow.
  - Simularea unui scenariu de echipă unde studenții trebuie să colaboreze pe același proiect, folosind pull requests pe GitHub pentru a revizui și accepta modificările.

7. Bune Practici în Git:
  - Discuții despre cele mai bune practici în utilizarea Git, inclusiv strategii pentru commit-uri eficiente, mesaje de commit clarificatoare, și gestionarea curată a branch-urilor.
  - Învățarea importanței și metodelor de backup și de a menține securitatea datelor în Git.

Concluzii:
- Laboratorul se încheie cu o recenzie a celor învățate și discuții despre cum Git poate fi integrat în fluxurile de lucru zilnice ale studenților și profesioniștilor în software.
- Studenții sunt încurajați să continue practicarea și explorarea funcționalităților Git pentru a-și îmbunătăți competențele de control al versiunilor.

Concepte Esențiale din Laboratorul 4: Utilizarea Git

1. Repository Git (Repo): Locul unde sunt stocate toate fișierele proiectului, istoricul lor și alte informații referitoare la modificări. Există două tipuri de repository-uri: locale și remote (de exemplu, pe GitHub).

2. Commit: Un "snapshot" al stării proiectului la un moment dat. Commit-urile stochează informații despre modificările făcute față de starea anterioară și sunt identificate printr-un hash SHA-1 unic.

3. Branch: O versiune independentă a istoricului repository-ului. Branch-urile permit dezvoltatorilor să lucreze pe funcționalități separate fără a influența codul principal (branch-ul "master" sau "main").

4. Merge: Procesul de combinare a modificărilor din două branch-uri diferite. Merge-ul poate fi automat, dacă nu există conflicte, sau poate necesita intervenția manuală.

5. Conflicte: Apar când același fragment de cod a fost modificat în două branch-uri diferite și Git nu poate decide automat care versiune să păstreze.

Comenzi Git Fundamentale

- `git init`: Inițializează un nou repository Git local.
- `git clone [url]`: Clonează un repository remote pe sistemul local.
- `git add [file]`: Adaugă fișiere în staging area înainte de commit.
- `git commit -m "[descriptive message]"`: Creează un nou commit cu fișierele din staging area.
- `git status`: Arată starea curentă a repository-ului, inclusiv modificările care nu au fost adăugate sau commit-uite.
- `git push [remote-name] [branch-name]`: Împinge commit-urile din branch-ul local în branch-ul remote.
- `git pull [remote-name] [branch-name]`: Trage ultimele modificări de pe remote și le combină cu branch-ul local.
- `git branch`: Listează toate branch-urile locale sau creează un nou branch.
- `git checkout [branch-name]`: Schimbă branch-ul curent la cel specificat.
- `git merge [branch]`: Combina modificările din branch-ul specificat în branch-ul curent.

Laborator 5: Testare Unitară în Python cu Pytest

Obiective:
- Introducerea conceptelor de testare unitară și a importanței acesteia în dezvoltarea software.
- Învățarea utilizării pytest pentru crearea și rularea testelor unitare în proiectele Python.

Activități Principale:

1. Introducere în Testarea Unitară:
   - Discuții despre ce este testarea unitară și de ce este esențială în ciclurile de dezvoltare ale software-ului pentru identificarea și corectarea timpurie a bug-urilor.
   - Explorarea principiilor de bază ale testării unitare, incluzând izolarea testelor, folosirea aserțiunilor și a fixture-urilor.

2. Configurarea Mediu de Lucru:
- Instrucțiuni pentru instalarea pytest și configurarea unui mediu de dezvoltare adecvat pentru testare.
- Crearea unui proiect simplu de Python și structurarea directorului pentru teste.

3. Scrierea Primelor Teste cu Pytest:
- Demonstrarea scrierii testelor unitare simple utilizând pytest, subliniind sintaxa și convențiile specifice.
- Implementarea unor teste pentru funcții Python simple, folosind aserțiuni pentru a verifica corectitudinea rezultatelor.

4. Utilizarea Fixtures pentru Setup și Teardown:
- Explicarea conceptului de fixtures în pytest, care facilitează setup-ul și teardown-ul necesar înainte și după fiecare test.
- Crearea și utilizarea fixtures pentru a gestiona resursele utilizate în teste, cum ar fi conexiuni la baze de date sau configurarea obiectelor complexe.

5. Teste Avansate și Parametrizare:
- Introducerea tehnicilor avansate de testare, inclusiv parametrizarea testelor pentru a rula aceeași funcție de test cu diferite valori de intrare.
- Explorarea utilizării markerelor în pytest pentru a controla comportamentul testelor, cum ar fi sărind peste teste sau așteptându-se la eșecuri.

6. Practici Recomandate în Testarea Unitară:
- Discuții despre cele mai bune practici în testarea unitară, inclusiv scrierea testelor curate și bine documentate.
- Sublinierea importanței testării continue și integrării acestora în procesele de integrare continuă (CI).

Concluzii:
- Laboratorul încheie cu o sesiune de întrebări și răspunsuri, unde studenții pot discuta dificultățile întâmpinate și pot primi feedback direct pe teste.
- Studenții sunt încurajați să continue explorarea capabilităților pytest și să integreze practicile de testare unitară în proiectele lor.

Concepte Esențiale din Laboratorul 5: Testare Unitară cu Pytest

1. Testarea Unitară:
- Definiție: Testarea unitară implică verificarea fiecărei componente sau unități de cod izolat pentru a se asigura că funcționează corect.
- Scop: Identificarea și corectarea erorilor în stadii incipiente ale dezvoltării, îmbunătățind calitatea codului și reducând costurile pe termen lung.

2. Pytest:

- Utilizare: O bibliotecă populară în Python pentru scrierea și rularea testelor unitare simplu și eficient.
- Beneficii: Suportă fixture-uri pentru setup și teardown, testare avansată cu parametrizare și utilizare ușoară a aserțiunilor.

3. Fixture-uri:
- Rol: Facilitarea setării și curățării condițiilor necesare testelor, cum ar fi configurarea unei baze de date de test sau a unui context de aplicație.
- Implementare: Definite cu decoratorul `@pytest.fixture`, permit reutilizarea în multiple funcții de test.

4. Aserțiuni:
- Scop: Verificarea condițiilor așteptate în teste. Aserțiunile eșuate indică probleme în logica aplicației.

5. Parametrizarea:
- Utilitate: Permite rularea aceluiași test cu diferite seturi de date pentru a verifica diverse scenarii.
- Metodă: Folosind `@pytest.mark.parametrize`, se specifică numele variabilei și valorile pe care testul le va lua.

Comenzi Pytest Esențiale

1. Instalare Pytest:
```bash
pip install pytest
```

2. Rulare Teste:
- Rulați toate testele dintr-un director:
```bash
pytest
```
- Rulați teste specifice folosind expresii:
```bash
pytest -k "some_keyword"
```

3. Folosirea Fixture-urilor:
```python
@pytest.fixture
def some_fixture():
    # setup code
    yield some_data
```

```
    # teardown code
```
```

4. Parametrizarea Testelor:
```python
@pytest.mark.parametrize("input,expected", [(input1, expected1), (input2, expected2)])
def test_function(input, expected):
    assert function_to_test(input) == expected
```
```

Laborator 6: Tehnici Avansate de Testare Unitară cu Pytest

Obiective:
- Învățarea utilizării tehnicilor avansate de testare, inclusiv mock-uri și patch-uri pentru a simula dependențe.
- Aprofundarea cunoștințelor despre gestionarea testelor pentru sisteme complexe.

Activități Principale:

1. Recapitulare și Introducere în Testarea Avansată:
  - Recapitularea conceptelor de bază ale testării unitare discutate în laboratorul anterior.
  - Introducerea conceptelor de testare avansată, incluzând mock-uri, patch-uri și testare parametrizată.

2. Utilizarea Mock-urilor în Pytest:
  - Explicarea conceptului de mocking și a cazurilor în care este util.
  - Demonstrarea folosirii `unittest.mock` pentru a crea mock-uri care să simuleze comportamentul obiectelor externe în teste.

3. Aplicarea Patch-urilor în Teste:
  - Instruirea studenților cum să folosească `unittest.mock.patch()` pentru a înlocui obiecte în modulul care este testat.
  - Realizarea unor exerciții practice prin care studenții să aplice patch-uri peste funcții sau obiecte pentru a controla output-ul acestora în timpul testelor.

4. Parametrizarea Testelor:
  - Discuții despre beneficiile parametrizării testelor pentru a rula aceleași teste cu diferite seturi de date.
  - Utilizarea `@pytest.mark.parametrize` pentru a demonstra cum se pot executa teste multiple de la o singură funcție de testare.

5. Testarea Excepțiilor și a Erorilor:

- Învățarea cum să se testeze corect gestionarea excepțiilor în codul sursă.
- Scrierea testelor care verifică dacă anumite erori sunt ridicate așa cum este așteptat sub condiții specifice.

6. Integrarea Testelor în CI/CD:
- Explorarea modurilor în care testele unitare pot fi integrate într-un pipeline de integrare și livrare continuă (CI/CD).
- Discuții despre cum testarea automată sprijină dezvoltarea agilă și ajută la menținerea calității în ciclurile rapide de release.

Concluzii:
- Laboratorul încheie cu o discuție despre importanța testării continue și a mentenanței codului testat în proiecte de software în evoluție.
- Studenții sunt încurajați să folosească tehnicile învățate pentru a îmbunătăți testele existente și pentru a asigura că noile caracteristici sunt bine testate înainte de integrarea în baza de cod principală.

Concepte Esențiale:
1. Mock-uri:
- Utilizate pentru a simula comportamentul obiectelor externe sau complexe în teste, astfel încât testele să se concentreze exclusiv pe codul testat fără a depinde de resurse externe sau alte module ale sistemului.

2. Patch-uri:
- Tehnica de "patching" permite înlocuirea temporară a unei metode, funcții sau a unui obiect întreg în timpul testului. Acest lucru este util pentru a controla comportamentul modulelor în timpul testării, precum și pentru a împiedica efectuarea unor operațiuni costisitoare sau nedorite (cum ar fi apelurile de rețea).

3. Fixtures:
- În pytest, fixtures sunt folosite pentru a crea un context de testare; ele pregătesc datele de intrare, configurează starea sistemului sau alocă/dezalocă resursele necesare înainte și după fiecare test.

4. Parametrizarea:
- Permite rularea acelorași teste cu diferite seturi de date, mărind acoperirea testelor și eficiența în identificarea cazurilor de eroare.

Comenzi pytest relevante:
- `pytest`: Comanda de bază pentru a rula teste.
- `@pytest.fixture`: Decorator folosit pentru a defini un fixture.
- `@pytest.mark.parametrize(argnames, argvalues)`: Decorator pentru a rula un test cu diferite combinații de argumente.
- `@pytest.mark.skip`: Decorator pentru a sări peste un test anume.

- `@pytest.mark.xfail`: Indică faptul că un test este așteptat să eșueze.

Utilizarea mock-urilor și patch-urilor:
- `from unittest.mock import MagicMock, patch`: Importă clasele necesare pentru mocking și patching.
- Utilizarea `patch()` în context manager sau ca decorator pentru a înlocui temporar module sau metode.
- `MagicMock()` poate fi folosit pentru a crea un obiect mock care poate fi configurat să returneze anumite valori sau să ridice excepții când este invocat.

Exemplu de utilizare:
```python
import pytest
from unittest.mock import patch, MagicMock

def function_to_test():
    # O funcție care depinde de un serviciu extern.
    pass

def test_function_to_test():
    with patch('path.to.external.service') as mock_service:
        mock_service.return_value = "Expected result"
        result = function_to_test()
        assert result == "Expected result"

@pytest.fixture
def some_fixture():
    return some_setup()

@pytest.mark.parametrize("input,expected", [
    (1, 2),
    (2, 3),
    (3, 4)
])
def test_with_parameters(input, expected):
    assert input + 1 == expected
```

Laborator 8: Testare de Integrare și Strategii Incrementale

Obiective:

- Familiarizarea studenților cu conceptele și tehnicile de testare de integrare, esențiale pentru validarea interacțiunilor dintre modulele software.
- Învățarea și aplicarea strategiilor de testare incrementală pentru a facilita identificarea problemelor de integrare.

Activități Principale:

1. Introducere în Testarea de Integrare:
   - Discuții despre diferențele între testarea unitară și cea de integrare și importanța fiecărei abordări în asigurarea calității software.
   - Prezentarea obiectivelor principale ale testării de integrare: validarea funcționalității combinate a mai multor module sau sisteme.

2. Strategii de Testare Incrementală:
   - Explorarea diferitelor strategii de testare incrementală, cum ar fi bottom-up, top-down, și sandwich (mixt).
   - Discuții despre avantajele și dezavantajele fiecărei metode, și selecția strategiei adecvate în funcție de structura proiectului și dependențele dintre module.

3. Implementarea Testării Bottom-Up:
   - Învățarea și aplicarea testării bottom-up, care începe cu testarea modulelor la nivelul cel mai scăzut și progresează către modulele superioare.
   - Utilizarea driverelor pentru a simula modulele superioare în timpul testării modulelor de nivel inferior.

4. Implementarea Testării Top-Down:
   - Aplicarea testării top-down, care începe cu modulele de nivel superior și integrează treptat modulele de nivel inferior.
   - Utilizarea stub-urilor pentru a simula funcționalitatea modulelor de nivel inferior în timpul testării modulelor superioare.

5. Utilizarea Testelor Big Bang:
   - Discuții despre abordarea Big Bang, în care toate modulele sau componentele sunt integrate și testate simultan.
   - Analiza riscurilor asociate cu această metodă, inclusiv dificultatea de a identifica sursele erorilor din cauza complexității integrate.

6. Automatizarea Testelor de Integrare:
   - Explorarea uneltelor și tehnologiilor disponibile pentru automatizarea testelor de integrare.
   - Demonstrarea implementării testelor de integrare într-un mediu de integrare continuă (CI).

Concluzii:
- Laboratorul se încheie cu o sesiune de evaluare a strategiilor testate, discutând eficacitatea și limitările fiecărei abordări în contextul proiectelor reale.

- Studenții sunt încurajați să continue explorarea și rafinarea tehnicilor de testare de integrare, și să integreze cele mai bune practici în procesele lor de dezvoltare software.

Concepte Esențiale:

1. Testarea de Integrare:
  - Obiectivul principal este de a verifica interacțiunile și compatibilitatea între modulele sau componentele unui sistem software.

2. Strategii Incrementale de Testare:
  - Bottom-Up: Începe testarea de la nivelul cel mai scăzut al modulelor software și avansează către componentele superioare.
  - Top-Down: Începe testarea de la modulele de la nivel superior și avansează către cele inferioare, necesitând adesea folosirea stub-urilor pentru a simula modulele inferioare încă netestate.
  - Big Bang: Integrarea și testarea simultană a tuturor componentelor sau modulelor, mai puțin preferată datorită complexității identificării surselor erorilor.

Comenzi Importante:
Testarea de integrare nu implică neapărat comenzi specifice la fel cum ar fi în cazul unor unelte de dezvoltare software, dar în contextul utilizării unor unelte pentru automatizarea acestui proces, se pot considera următoarele comenzi într-un mediu de integrare continuă (CI), cum ar fi Jenkins sau GitHub Actions:

- Jenkins:
  ```bash
  # Comandă pentru declanșarea unui build/test:
  java -jar jenkins-cli.jar -s http://yourjenkinsserver/ build your-job-name
  ```

- GitHub Actions:
  - Fiecare workflow în GitHub Actions este definit într-un fișier YAML în directorul `.github/workflows`. Exemplu de comandă de testare într-un workflow:
    ```yaml
    jobs:
      build:
        runs-on: ubuntu-latest
        steps:
        - uses: actions/checkout@v2
        - name: Run integration tests
          run: make integration-test
    ```
  - Aceasta comandă ar declanșa testele de integrare definite în Makefile-ul proiectului.

Laborator 7: Automatizarea CI/CD cu GitHub Actions

Obiective:
- Familiarizarea studenților cu conceptele și beneficiile integrării continue (CI) și livrării continue (CD).
- Învățarea configurării și utilizării GitHub Actions pentru a automatiza testarea, construirea și deploy-ul aplicațiilor.

Activități Principale:

1. Introducere în CI/CD și GitHub Actions:
   - Prezentarea fundamentelor CI/CD și discuții despre rolul acestora în ciclul de dezvoltare modern.
   - Introducerea GitHub Actions ca unealtă pentru automatizarea CI/CD, explicând conceptul de workflows și jobs.

2. Configurarea Primului Workflow:
   - Crearea unui nou repository GitHub și scrierea primului workflow folosind YAML.
   - Instrucțiuni pas cu pas pentru configurarea unui workflow simplu care să execute testele unitare la fiecare push în branch-ul principal.

3. Automatizarea Testelor și Construirii:
   - Extinderea workflow-ului pentru a include nu doar testarea ci și construirea (build) aplicației.
   - Explorarea acțiunilor predefinite disponibile în marketplace-ul GitHub pentru diverse limbaje de programare și unelte de build.

4. Implementarea Livrării Continue (CD):
   - Adăugarea pașilor în workflow pentru deploy automat, folosind acțiuni de tipul "deploy to Azure" sau "deploy to AWS".
   - Discuții despre managementul secretelor și variabilelor de mediu în GitHub Actions pentru a securiza datele sensibile.

5. Testarea și Depanarea Workflows:
   - Învățarea tehnicilor de depanare a workflows, inclusiv verificarea logurilor și ajustarea configurărilor în caz de erori.
   - Simularea unei erori în cod pentru a vedea cum reacționează workflow-ul și cum pot fi identificate și rezolvate problemele.

6. Best Practices și Strategii Avansate:

- Discuții despre cele mai bune practici în configurarea și optimizarea workflows în GitHub Actions.
- Explorarea strategiilor avansate, cum ar fi matrix builds pentru a rula teste pe multiple platforme și versiuni simultan.

Concluzii:
- Laboratorul se încheie cu o revizuire a proiectelor studenților, oferind feedback individual și sugestii pentru îmbunătățiri.
- Studenții sunt încurajați să exploreze alte unelte și platforme de CI/CD, și să integreze practicile învățate în proiectele lor viitoare.

Concepte Esențiale din Laboratorul 7: Automatizarea CI/CD cu GitHub Actions

1. Integrarea Continuă (CI):
- Definiție: Proces automatizat prin care codul dezvoltat este frecvent integrat într-un repository comun, unde sunt executate teste automate pentru a detecta erori cât mai devreme posibil.

2. Livrarea Continuă (CD):
- Definiție: Extensie a integrării continue, asigurând că orice schimbare validată prin teste poate fi automat lansată în producție printr-un proces automatizat, permițând livrări rapide și fiabile.

3. GitHub Actions:
- Definiție: Serviciu de automatizare care permite definirea și rularea workflow-urilor direct dintr-un repository GitHub. Acestea pot gestiona diverse taskuri, de la testare și construire până la deployment.

4. Workflows:
- Definiție: Procese automate definite printr-un fișier YAML în directorul `.github/workflows` al unui repository. Workflows pot rula bazat pe evenimente specifice, cum ar fi push-uri, pull requests sau programări.

5. Jobs și Steps:
- Definiție: Un workflow conține unul sau mai multe jobs care sunt seturi de taskuri ce rulează pe același runner. Fiecare job conține steps, care pot rula comenzi sau acțiuni.

Comenzi Principale și Sintaxă GitHub Actions

1. Crearea unui Workflow:
- Un workflow este definit într-un fișier YAML în directorul `.github/workflows` al repository-ului. Exemplu de structură a unui fișier workflow pentru un proiect simplu de Python care rulează teste:

```yaml
name: Python application test

on: [push]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v2
    - name: Set up Python 3.x
      uses: actions/setup-python@v2
      with:
        python-version: '3.x'
    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt
    - name: Run tests
      run: |
        python -m pytest
```

2. Comenzi Uzuale în Steps:
   - `actions/checkout@v2`: Această acțiune configurează repository-ul Git în runner, permitând accesul la cod.
   - `actions/setup-python@v2`: Configurează un mediu Python pe runner, necesar pentru rularea scripturilor Python.
   - `run`: Execută comenzi direct în shell-ul runner-ului.

3. Automatizarea Deployment-ului:
   - Adăugarea pașilor de deploy în același workflow sau într-un workflow separat, care poate include configurarea și utilizarea serviciilor cloud, containerizare, etc.

Laborator 9: Testarea Securității Software cu Fuzzing

Obiective:
- Familiarizarea studenților cu importanța testării de securitate în ciclul de dezvoltare software.
- Învățarea utilizării tehnicilor de fuzzing pentru a identifica și remedia vulnerabilitățile din software.

Activități Principale:

1. Introducere în Testarea de Securitate:
   - Discuții despre conceptele de bază ale securității software și rolul testării de securitate în prevenirea incidentelor.
   - Prezentarea diferitelor tipuri de teste de securitate, inclusiv testele statice, dinamice și de penetrare.

2. Ce este Fuzzing și Cum Funcționează:
   - Explorarea conceptului de fuzzing ca metodă de testare care generează intrări aleatorii (sau semi-aleatorii) pentru a descoperi erori și vulnerabilități.
   - Discuții despre diferitele tipuri de fuzzing, de la fuzzing negru (black-box) la fuzzing bazat pe cunoașterea structurii interne (white-box).

3. Configurarea și Utilizarea unui Fuzzer:
   - Pas cu pas, configurarea unui fuzzer simplu utilizând unelte disponibile, cum ar fi AFL (American Fuzzy Lop) sau LibFuzzer.
   - Practica de a rula fuzzer-ul pe exemple de cod pentru a vedea cum se comportă și ce tipuri de erori pot fi descoperite.

4. Analiza Rezultatelor și Remedierea:
   - Învățarea interpretării rezultatelor testării fuzzing, identificarea vulnerabilităților și a bug-urilor.
   - Discuții despre strategii pentru remedierea problemelor descoperite, inclusiv corectarea codului și îmbunătățirea testelor.

5. Integrarea Fuzzing în CI/CD:
   - Explorarea modalităților de integrare a testelor de fuzzing în procesele de integrare continuă (CI) și livrare continuă (CD) pentru a automatiza testarea de securitate.
   - Discuții despre beneficiile și provocările implementării fuzzing-ului ca parte din pipeline-ul de CI/CD.

6. Best Practices și Considerații Etice:
   - Prezentarea celor mai bune practici în testarea de securitate cu fuzzing, inclusiv gestionarea datelor sensibile și configurarea adecvată a testelor.
   - Considerații etice legate de testarea de securitate, cum ar fi comunicarea transparentă și responsabilă a vulnerabilităților descoperite.

Concluzii:
- Laboratorul se încheie cu o sesiune de întrebări și răspunsuri, unde studenții pot discuta despre provocările întâmpinate și pot primi feedback direct.
- Studenții sunt încurajați să continue explorarea și aplicarea tehnicilor de testare de securitate în proiectele lor viitoare.

Concepte Esențiale din Laboratorul 9: Testarea Securității Software cu Fuzzing

Testarea de Securitate:
- Importanța Testării de Securitate: Esențială pentru identificarea și remedierea vulnerabilităților în software înainte de a fi exploatate de atacatori.
- Fuzzing: O metodă de testare care automatizează introducerea datelor aleatorii sau semi-aleatorii în aplicații pentru a detecta erori și vulnerabilități.

Tipuri de Fuzzing:
- Black-box Fuzzing: Fuzzing fără cunoștințe despre structura internă a programului, focalizat pe interfața utilizatorului și intrările externe.
- White-box Fuzzing: Bazat pe analiza codului sursă pentru a crea intrări care acoperă cât mai mult cod posibil, descoperind astfel vulnerabilitățile ascunse.

Comenzi și Utilizare în Contextul Fuzzing

1. Configurarea și Utilizarea AFL (American Fuzzy Lop):
  - Instalare AFL:
    ```
    sudo apt-get install afl
    ```
  - Compilarea Programului pentru Fuzzing:
    ```
    afl-gcc -o program_fuzzed program.c
    ```
  - Rularea AFL pe Program:
    ```
    afl-fuzz -i input_dir -o findings_dir ./program_fuzzed
    ```

2. Utilizarea LibFuzzer pentru Fuzzing Integrat în Cod:
  - Adăugarea Suportului LibFuzzer:
    - Codul sursă trebuie să includă hook-uri pentru LibFuzzer, de obicei sub forma unei funcții `LLVMFuzzerTestOneInput`.
  - Compilarea cu LibFuzzer:
    ```
    clang -fsanitize=fuzzer,address -o my_fuzzer my_fuzzer.c
    ```
  - Rularea Fuzzer-ului:
    ```
    ./my_fuzzer
    ```

Integrația Fuzzing în CI/CD

- Automatizarea Fuzzing: Integrarea testelor de fuzzing în pipeline-urile de CI/CD poate fi realizată prin adăugarea unui job de testare în configurația CI care rulează fuzzer-ul și raportează rezultatele.
- Monitorizarea și Analiza Rezultatelor: Este crucial să se monitorizeze rezultatele fuzzing-ului și să se analizeze bug-urile sau vulnerabilitățile descoperite pentru remedierea rapidă.

Laborator 10: Optimizarea Performanței și Scalabilitatea Aplicațiilor Software

Obiective:
- Familiarizarea studenților cu tehnicile de optimizare a performanței software.
- Învățarea metodelor de scalare a aplicațiilor pentru a susține creșterea cerințelor de sistem și utilizator.

Activități Principale:

1. Introducere în Performanța Software:
   - Discuții despre importanța performanței în dezvoltarea software și impactul acesteia asupra experienței utilizatorului.
   - Prezentarea factorilor care influențează performanța, inclusiv eficiența codului, arhitectura sistemului și resursele hardware.

2. Tehnici de Profilare și Monitorizare a Performanței:
   - Învățarea utilizării uneltelor de profilare pentru a identifica gâturile de sticlă în aplicații.
   - Practica de monitorizare a performanței în timp real folosind unelte precum New Relic sau Prometheus.

3. Optimizarea Codului:
   - Discuții și exerciții practice legate de optimizarea codului pentru a îmbunătăți timpul de răspuns și eficiența resurselor.
   - Implementarea tehnicilor de caching, refactorizarea algoritmilor și reducerea complexității operațiilor.

4. Scalabilitatea Aplicațiilor:
   - Explorarea conceptelor de scalabilitate orizontală și verticală și când să aplici fiecare metodă.
   - Exemplificarea scalabilității prin adăugarea de instanțe de server sau prin îmbunătățirea specificațiilor hardware.

5. Utilizarea Microserviciilor pentru Scalare:
   - Discuții despre arhitectura bazată pe microservicii și beneficiile acesteia pentru scalabilitate și mentenanță.

- Implementarea unui exemplu simplu de microservicii utilizând Docker și Kubernetes pentru a demonstra distribuirea sarcinilor.

6. Evaluarea și Testarea Scalabilității:
- Învățarea cum să testezi și să validezi scalabilitatea aplicațiilor folosind teste de încărcare și stres.
- Utilizarea uneltelor automate pentru simularea traficului mare și evaluarea performanței sub sarcină.

Concluzii:
- Laboratorul se încheie cu o discuție despre cum optimizarea continuă și atenția la scalabilitate pot influența succesul pe termen lung al unui produs software.
- Studenții sunt încurajați să aplice cunoștințele dobândite în proiectele lor, cu scopul de a dezvolta aplicații robuste, eficiente și scalabile.

Concepte Esențiale din Laboratorul 10: Optimizarea Performanței și Scalabilitatea Aplicațiilor Software

Concepte Esențiale:
1. Performanța Software:
- Importanța performanței pentru experiența utilizatorului și eficiența operațională.
- Factori cheie care influențează performanța: eficiența codului, arhitectura sistemului, resurse hardware.

2. Optimizarea Codului:
- Tehnici pentru îmbunătățirea eficienței codului: caching, refactorizarea algoritmilor, reducerea complexității operațiilor.

3. Scalabilitatea Aplicațiilor:
- Scalabilitate orizontală (adaugarea de instanțe) vs. scalabilitate verticală (îmbunătățirea resurselor unui server).
- Utilizarea arhitecturii bazate pe microservicii pentru a îmbunătăți scalabilitatea și mentenanța.

Comenzi și Unelte Tehnice:
1. Profilare și Monitorizare:
- Utilizarea uneltelor de profilare:
  ```
  python -m cProfile -o output_file.prof your_script.py
  ```
- Monitorizarea performanței cu New Relic sau Prometheus pentru a observa metrici în timp real.

2. Implementarea Scalabilității cu Microservicii:

- Utilizarea Docker pentru containerizare:
  ```
  docker build -t your-app .
  docker run -p 4000:80 your-app
  ```
- Implementarea microserviciilor cu Kubernetes:
  ```
  kubectl apply -f deployment.yaml
  kubectl get pods
  ```

3. Testarea Scalabilității:
   - Utilizarea uneltelor de testare a încărcării cum ar fi Apache JMeter sau Locust:
     ```
     jmeter -n -t your_test_plan.jmx -l test_results.jtl
     locust -f locustfile.py --host=http://your-application.com
     ```